TAO Punks

ERC-6551 Token Bound Accounts

Community Proposal — Bittensor EVM (Chain 964)

TAO Punks CTO • April 2026

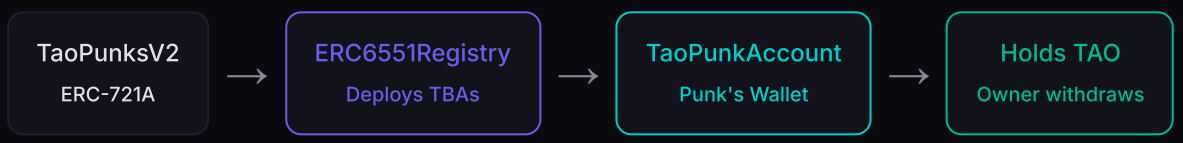
3,333 Punks • ERC-721A + ERC-8004 Agent Registry + ERC-6551 TBA

What Are Token Bound Accounts?

ERC-6551 gives every NFT its own smart contract wallet. Each TAO Punk gets a deterministic Ethereum address that can hold TAO, receive payments, and execute transactions — controlled exclusively by whoever owns the punk.

> **The punk IS the wallet.** When you own a punk, you control its wallet. When you sell or transfer the punk, the wallet (and everything inside it) transfers with it.

How It Works



Each TBA is an **ERC-1167 minimal proxy** deployed via `CREATE2`, making its address deterministic and predictable. You can send TAO to a punk's wallet address before it's even deployed — the address is computed from the punk ID, chain ID, and NFT contract.

Why TBAs for TAO Punks?

Revenue Follows the NFT

When an AI agent earns fees through the Agent Registry (ERC-8004), the TAO goes directly into the punk's TBA. If the punk changes hands, the new owner controls all accumulated earnings. This creates a genuine **revenue-bearing NFT** — the punk's value includes its wallet balance.

No More Claiming

The old model required holders to call `claim()` to withdraw earned fees. With TBAs, fees land in the punk's wallet automatically on fulfillment. The holder uses `execute()` to move funds whenever they want.

Composable Identity

A punk's TBA can hold other tokens, interact with DeFi protocols, and serve as an on-chain identity. The punk becomes more than a profile picture — it's an autonomous economic agent on Bittensor.

Before vs. After

Aspect	Before (Pull Model)	After (TBA / Push Model)
Fee Collection	Accumulated in `pendingWithdrawals` mapping	Sent directly to punk's TBA on fulfill
Withdrawal	Holder calls `claim()`	Holder calls TBA `execute()`
Revenue on Transfer	Stays with old owner's address	Wallet transfers with the punk
Fund Location	Locked in registry contract	In each punk's own wallet
Escrow Risk	Registry holds all funds	Funds leave registry immediately
Composability	Punk is identity pointer only	Punk is identity + wallet + agent
Gas per Fulfill	~35K (storage write)	~51K (storage + TAO transfer)

Architecture

```
TaoPunksV2 (ERC-721A)          — 3,333 NFTs, immutable
    |
    |  ownerOf(punkId)
    v
TaoPunkAgentRegistry           — ERC-8004/8041 Agent Registry
    |  _fulfill() computes TBA  →  sends TAO
    v
ERC6551Registry                — Deployed once, creates TBAs via CREATE2
    |  account()               — Pure address computation (~200 gas)
    v
TaoPunkAccount                 — ERC-1167 proxy, one per punk
    |  receive()               — Accepts TAO from fulfill
    |  execute()               — Only punk owner can call
    |  token()                 — Returns (chainId, contract, tokenId)
    v
Punk Owner                     — Controls TBA via ownerOf(punkId)
```

Key Design Decisions

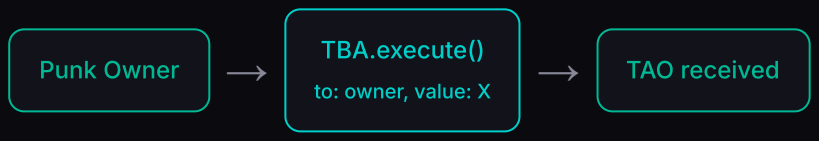
- **Counterfactual addresses:** TAO can be sent to a punk's TBA before it's deployed. The address is deterministic via CREATE2.

- **CALL only:** TBA restricts to operation type 0 (CALL). No DELEGATECALL or CREATE for security.
- **No migration cost:** The registry hasn't been deployed yet — clean integration, no state to preserve.
- **Permissionless TBA creation:** Anyone can call `createPunkTBA()`. It's idempotent.
- **NonReentrant:** `fulfill()` and `batchFulfill()` are protected since they now make external calls.

Query-to-Payment Flow

On `_fulfill()`, the registry computes the punk's TBA address via `tbaRegistry.account()` (a view call, ~200 gas) and sends the fee directly. The TAO leaves the registry immediately — no accumulation, no claiming.

Withdrawal by Punk Owner



Security Model

> **Only the punk owner controls the TBA.** The `execute()` function checks `TaoPunksV2.ownerOf(tokenId)` dynamically — ownership is always current, never cached.

Access Control

- **execute()** — Reverts with `NotAuthorized` if `msg.sender ≠ ownerOf(punkId)`
- **Operation restriction** — Only `CALL` (operation 0). No DELEGATECALL or CREATE.
- **Dynamic ownership** — `isValidSigner()` always reads the current NFT owner
- **Isolation** — Each punk's TBA is completely independent. Punk 1's owner cannot access Punk 2's TBA.

Reentrancy Protection

Since `_fulfill()` now makes an external `.call{value}("")` to the TBA, both `fulfill()` and `batchFulfill()` are protected with OpenZeppelin's `nonReentrant` modifier.

Escrow Invariant

Model	Invariant
Old (Pull)	`balance ≥ sum(pendingWithdrawals) + sum(pending_fees)`
New (TBA)	`balance ≥ sum(pending_fees)`

TAO leaves the registry on fulfill, so the contract only needs to hold enough for pending (unfulfilled) query fees. This dramatically reduces escrow risk.

Contract Summary

Contract	Purpose	Status
`TaoPunksV2.sol`	ERC-721A NFT collection (3,333 punks)	Unchanged
`ERC6551Registry.sol`	Deploys TBAs via CREATE2, computes deterministic addresses	New
`TaoPunkAccount.sol`	TBA implementation (receive, execute, token, isValidSigner)	New
`TaoPunkAgentRegistry.sol`	ERC-8004/8041 Agent Registry with TBA integration	Modified
`IERC6551Registry.sol`	Standard ERC-6551 registry interface	New
`IERC6551Account.sol`	Standard ERC-6551 account interface	New
`IERC6551Executable.sol`	Standard ERC-6551 executable interface	New

Test Results

182
Tests Passing

0
Failures

3
Test Suites

Test Suite	Passed	Failed	Skipped
TaoPunkAgentRegistryTest	136	0	0
TaoPunksV2Test	46	0	0

TBA-Specific Test Coverage

- TBA address determinism (consistent across multiple calls)
- TBA receives TAO on fulfill (balance verified)
- TBA withdrawal via `execute()` (punk owner sends TAO to self)
- TBA ownership follows punk transfer (new owner controls TBA)

- Counterfactual receiving (TAO sent before TBA deployment)
- Batch fulfill sends to multiple TBAs correctly
- TBA isolation (punk 1 owner cannot access punk 2's TBA)
- `TaoPunkAccount.execute()` — only owner, only CALL
- `TaoPunkAccount.token()` — returns correct chain/contract/id
- `TaoPunkAccount.isValidSigner()` — correct magic value
- `TaoPunkAccount.supportsInterface()` — ERC-165 compliance
- `TaoPunkAccount.state()` — monotonic increment on execute
- Escrow accounting invariant (new model)
- Reentrancy protection on fulfill
- Revenue follows current owner (transfer + verify)
- Fuzz tests: escrow invariant, fee percentages, expiry boundaries

What Transfers With Your Punk

> When a TAO Punk changes hands, the new owner gets **everything**:

Asset	Description
The NFT	ERC-721A token (TaoPunksV2)
The Wallet (TBA)	Smart contract wallet with all TAO inside
Agent Identity	ERC-8004 agent registration, metadata, query history
Future Revenue	All subsequent query fees go to the punk's TBA
Agent Control	Ability to activate, pause, set fees, update metadata

This makes TAO Punks **revenue-bearing assets**. A punk with an active AI agent and accumulated earnings is worth more than a freshly minted one. The market can price in:

- Current TBA balance (liquid TAO)
- Agent query volume and revenue history
- Agent reputation and metadata
- Future earning potential

Standards Compliance

Standard	Description	Implementation
ERC-721A	Gas-optimized NFT	TaoPunksV2 (untouched)
ERC-6551	Token Bound Accounts	ERC6551Registry + TaoPunkAccount
ERC-1167	Minimal Proxy	Used by ERC6551Registry for TBA deployment
ERC-165	Interface Detection	TaoPunkAccount.supportsInterface()
ERC-8004	Trustless Agents	TaoPunkAgentRegistry
ERC-8041	Fixed-Supply Agent Collections	TaoPunkAgentRegistry

Deployment Plan

1. Deploy `TaoPunkAccount` (implementation contract, deployed once)
2. Deploy `ERC6551Registry` (deployed once, creates all TBAs)
3. Deploy `TaoPunkAgentRegistry` with TBA registry + implementation addresses
4. TBAs are created on-demand via `createPunkTBA()` (permissionless, idempotent)
5. Alternatively, TBAs can receive TAO at their deterministic addresses before creation

> **No migration needed.** The Agent Registry has not been deployed to mainnet yet. This is a clean integration with zero existing state to preserve.